Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Дискретна математика

Лабораторна робота №2

«Бінарні відношення та їх основні властивості, операції над відношеннями».

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 18
Перевірив:
Пономаренко А. М.

Київ – 2024

Лабораторна робота №2

Тема: «Бінарні відношення та їх основні властивості, операції над відношеннями».

Мета: вивчити основні властивості бінарних відношень та оволодіти операціями над бінарними відношеннями.

Загальне завдання:

1. Написати в окремому модулі функцію для формування несуперечливих бінарних відношень.
2. Написати в окремому модулі функції виконання логічних операцій над бінарними відношеннями.
3. Пояснити правило формування несуперечливих відношень відповідно до Вашого варіанту.

Теоретичні основи:

Бінарним відношенням R називають підмножину впорядкованих пар, тобто множину, кожен елемент якої є впорядкованою парою. Якщо R є деяким відношенням, це записують як $(x, y) \in R$ або xRy

Один з типів відношень є множина всіх таких пар *(x, y)*, де x є елементом деякої фіксованої множини X, а y − елементом деякої фіксованої множини Y. Таке відношення називають прямим або декартовим добутком.

Декартовим добутком X×Y множин X і Y називають *множину* $\{(x, y) \mid x \in X, y \in Y\}$. При цьому, множину X називають областю визначення відношення R, а Y – його областю значень:

$$D(R) = \{x \mid (x, y) \in R\}; \quad E(R) = \{y \mid (x, y) \in R\}$$

Бінарне відношення R є підмножиною пар (x, y) прямого добутку X×Y, тобто $R \subseteq X \times Y$.

У силу визначення бінарних відношень, як **спосіб їх задавання** можуть бути використані будь-які способи задавання множин. Відношення, визначені на скінченних множинах, зазвичай задають такими способами:

1. Списком (перерахуванням) упорядкованих пар, для яких виконується це відношення.
2. Матрицею – бінарному відношенню $R \subseteq X \times X$, де $X = \{x_1; x_2; \dots; x_n\}$ відповідає квадратна матриця порядку n, кожен елемент a_{ij} якої дорівнює 1, якщо між x_i й x_j є відношення R, і 0 у протилежному випадку, тобто:

$$a_{ij} = \begin{cases} 1, & \text{якщо } x_i R x_j, \\ 0, & \text{у протилежному випадку.} \end{cases}$$

Властивості бінарних відношень є такими:

1. Відношення R на $A \times A$ називають **рефлексивним**, якщо $(a, a) \in R$ для кожного $a \in A$. Головна діагональ матриці такого відношення містить тільки одиниці.
2. Відношення R на $A \times A$ називають **антирефлексивним**, якщо для жодного $a \in A$ не виконується $(a, a) \in R$, тобто із $(a, b) \in R$ потрібно, щоб $a \neq b$. Головна діагональ матриці такого відношення містить тільки нулі.
3. Відношення R на $A \times A$ називають **симетричним**, якщо для всіх $(a, b) \in R$ з умови $(a, b) \in R$ потрібно, щоб $(b, a) \in R$. Матриця симетричного відношення симетрична відносно головної діагоналі, тобто $c_{ij} = c_{ji}$ для всіх i і j .
4. Відношення R на $A \times A$ називають **антисиметричним**, якщо для всіх $(a, b) \in R$, з умов $(a, b) \in R$ і $(b, a) \in R$ потрібно, щоб $a = b$, тобто для жодних елементів a і b, що розрізняються ($a \neq b$), не виконуються одночасно відношення $(a, b) \in R$ і $(b, a) \in R$. У матриці антисиметричного відношення відсутні одиниці, симетричні відносно головної діагоналі.
5. Відношення R на $A \times A$ називають **транзитивним**, якщо для будь-яких a, b, c з умов $(a, b) \in R$ і $(b, c) \in R$ випливає $(a, c) \in R$. У матриці такого відношення повинна виконуватися наступна умова: якщо в i-тому рядку і в j-тому стовпці стоїть одиниця, тобто $c_{ij} = 1$, то всім одиницям в j -тому рядку і k -тому стовпці ($c_{jk} = 1$) повинні відповідати одиниці в i -тому рядку і у тих же k -тих стовпцях, тобто $c_{ik} = 1$ (і, можливо, в інших стовпцях).
6. Бінарне відношення називають **еквівалентним**, якщо воно рефлексивне, симетричне і транзитивне.

Оскільки відношення на множині A задають підмножинами $R \subseteq A \times B$, то для них визначні ті ж операції, що й над множинами, а саме:

1. **Об'єднання:** $R_1 \cup R_2 = \{(a, b) \mid (a, b) \in R_1 \text{ або } (a, b) \in R_2\}$

2. Перетин: $R_1 \cap R_2 = \{(a,b) \mid (a,b) \in R_1 \text{ і } (a,b) \in R_2\}$

3. Різниця: $R_1 \setminus R_2 = \{(a,b) \mid (a,b) \in R_1 \text{ і } (a,b) \notin R_2\}$

4. Доповнення: $\bar{R} = U \setminus R =$ де $U = A \times B$

Крім того, необхідно визначити інші операції над бінарнимими відношеннями:

5. Обернене відношення R^{-1}: Якщо (a, b) ∈ R – відношення, то відношення R^{-1} називають **оберненим відношенням** до даного відношення R тоді і тільки тоді, коли $R^{-1} = \{(b,a) \mid (a,b) \in R\}$
6. **Композицією відношень R і S** (нехай R ⊆ A×B, S ⊆ B×C), називають відношення T ⊆ A×C, таке що $T = \{(a,c) \mid \text{існує такий } b \text{ з } B,$ що $(a,b) \in R$ і $(b,c) \in S\}$. Цю множину позначають $T = S \circ R$.

   Зокрема, якщо відношення R визначене на множині A ($R \subseteq A^2$), то композицію визначають як

$$R \circ R = R^{(2)} = \{(a,c) | (a,b),(b,c) \in R\}$$

7. **Транзитивним замиканням** R^o називають множину, що складається з таких і тільки таких пар елементів a, b з A, тобто (a,b) ∈ R^o, для яких в A існує ланцюжок з n+2 (n ≥ 0) елементів A: a, c_1, c_2, … c_n, b між сусідніми елементами якої виконується відношення R:(a, c_1) ∈ R, (c_1, c_2) ∈ R, ..., (c_n, b) ∈ R, тобто

$$R^o = \{(a,b) | (a,c_1),(c_1,c_2),\dots,(c_n,b) \in R\}$$

   Якщо відношення R транзитивне, то R^o = R. Транзитивне замикання R^o на R є найменшим транзитивним відношенням на A, що містить R як підмножину.

8. **Рефлексивне замикання:**

   Нехай тотожне відношення E = {(a, a)| a ∈ A}. Тоді рефлексивне замикання визначають як R* = R^o ∪ E. Якщо R транзитивне і рефлексивне, то R* = R. Рефлексивне замикання R* на R є найменшим рефлексивним відношенням на A, що містить R як підмножину.

Загальний порядок виконання роботи:

A. Вибрати номер Z індивідуального варіанта відповідно до виразу: *Z=(i+G%60)%30+1*, де i – номер у списку групи, G – числова складова назви групи.

Програма обчислення варіанта:

```
G=64
N=1
print("Моя група: ІО -",G)
print("Мій номер у групі:",N)
print("Мій варіант:",(N+G%60)%30+1)
```

B. За індивідуальним варіантом побудувати алгоритм, який встановлює одночасно несуперечливі відношення S та R для двох довільних множин.

C. Відношення S та R представити у вигляді матриць або графів.

D. Результат операцій над відношеннями також представити у вигляді матриць або графів.

Вимоги до інтерфейсу:

Програма повинна складатися з 4-ьох вікон.

1) **Вміст вікна №1.**
   1. Головне меню, яке повинно включати виклик вікна №2, вікна №3 та вікна №4.
   2. Віджети виводу П.І.Б студента, номера групи, номера у списку та віджет виводу результатів обчислення варіанту відповідно да програми, що задана у завданні – (пункт 1 загального порядку виконання лабораторної роботи).
2) **Вміст вікна №2**
   1. Віджет Listbox зі списком жіночих імен з можливістю вибору з нього довільної кількості імен з наступним копіюванням цих імен у віджет Listbox списку множини A або B, в залежності від стану віджета Radiobutton, який вказує у який список відбувається копіювання імені.
   2. Віджет Listbox зі списком чоловічих імен з можливістю вибору з нього довільної кількості імен з наступним

копіюванням цих імен у віджет Listbox списку множини A або B, в залежності від стану віджета Radiobutton, який вказує у який список відбувається копіювання імені.

3. Віджети Button для збереження у файл, зчитування з файлу та очищення множини A.
4. Віджети Button для збереження у файл, зчитування з файлу та очищення множини B.

3) Вміст вікна №3

1. Віджети Listbox для відображення елементів множин A і B.
2. Матричне або графічне представлення відношення S, яке сформоване вашим алгоритмом
3. Матричне або графічне представлення відношення R, яке сформоване вашим алгоритмом з урахуванням несуперечливості з відношенням S.

4) Вміст вікна №4

1. Матричне або графічне представлення відношення $R \cup S$
2. Матричне або графічне представлення відношення $R \cap S$
3. Матричне або графічне представлення відношення $R \setminus S$
4. Матричне або графічне представлення відношення $U \setminus R$
5. Матричне або графічне представлення відношення S^{-1}

<u>Індивідуальне завдання:</u>

Завдання виконане згідно 18 варіанту.

№ вар	Відношення 1	Відношення 2
1	aSb, якщо a мати b	aRb, якщо a онука b

<u>Розрахунок варіанту:</u>

<u>Код програми:</u>

```
G=42
N=18
print("Моя група: IO -",G)
print("Мій номер у групі:",N)
print("Мій варіант:",(N+G%60)%30+1)
```

Результат виконання програми:

```
"D:\Documents\Education\KPI\2 Term\D
Моя група: IO - 42
Мій номер у групі: 18
Мій варіант: 1
```

Рисунок 1 – результат обрахунку варіанта

Алгоритми для створення відношень S та R:

Для початку обраховуємо декартовий добуток множин A та B.

Для формування відношення S (aSb, a є мати b): Шукаємо можливих жінок з перших елементів декартового добутку, які потенційно можуть бути матерями. Далі, на основі цього ми шукаємо можливих дітей, і враховуємо що у однієї дитини може бути одна матір. Утворюємо пари, враховуючи ці моменти.

Для формування відношення R (aRb, a онука b): враховуємо, що пари з відношення S не можуть бути у відношенні R, а також перший елемент з пари повинна бути жінкою. Варто розглянути деякі пари, де у дитини є мати, і у її мати є батьки, то ми можемо її додати у відношення R.

Код програми:

Вміст файлу LR2Data.py:

```
import random

female_names = ["Анастасія", "Аліса", "Ганна", "Галина", "Ірина", "Наталя", "Марія", "Ольга",
"Тетяна", "Катерина", "Людмила", "Вікторія", "Софія", "Дарина", "Юлія"]
male_names = ["Андрій", "Богдан", "Євген", "Дмитро", "Степан", "Микола", "Олег", "Василь",
"Роман", "Павло", "Іван", "Сергій", "Олександр", "Володимир", "Артем"]


set_a = set()
set_b = set()
cartesian_product = set()
s = set()
r = set()

#розрахунок варіанту
def calculate_var(group_fullnum, personal_num):
    return (group_fullnum + personal_num % 60) % 30 + 1

#декартовий добуток
def calculate_cartesian_product():
    if not set_a and not set_b:
        return
    cartesian_product.clear()
    for a in set_a:
        for b in set_b:
            cartesian_product.add((a, b))


#генерація відношення S => aSb a є мати b
def create_random_s():
```

```
    s.clear()
    calculate_cartesian_product()
    available_children = list(set_b)
    random.shuffle(available_children)
    mother_children = {mother: set() for mother in set_a if mother in female_names}
    for mother in mother_children:
        max_children = random.randint(0, 2)
        possible_children = [child for child in available_children
                            if (mother, child) in cartesian_product and child not in {c for i, c in s}]
        random.shuffle(possible_children)
        while possible_children and len(mother_children[mother]) < max_children:
            child = possible_children.pop()
            s.add((mother, child))
            mother_children[mother].add(child)
            available_children.remove(child)

#генерація відношення R => aRb a є онукою b
def create_random_r():
    r.clear()
    mother_of = {child: mother for mother, child in s}
    for grandchild, mother in mother_of.items():
        if mother in mother_of:
            grandparent = mother_of[mother]
            if grandchild in female_names and (grandchild, grandparent) in cartesian_product and
(grandchild, grandparent) not in s:
                r.add((grandchild, grandparent))

    if len(r) < 2:
        potential_grandchildren = [name for name in set_a if name in female_names]
        potential_grandparents = list(set_b)
        random.shuffle(potential_grandchildren)
        random.shuffle(potential_grandparents)
        max_grandparents = len(potential_grandparents)
        avg_pairs_per_grandchild = max(1, max_grandparents // len(potential_grandchildren))
        for grandchild in potential_grandchildren:
            num_pairs = max(2, random.randint(1, avg_pairs_per_grandchild))
            for i in range(num_pairs):
                if potential_grandparents:
                    grandparent = random.choice(potential_grandparents)
                    if (grandchild, grandparent) in cartesian_product and (grandchild, grandparent) not in s:
                        r.add((grandchild, grandparent))
                        potential_grandparents.remove(grandparent)

                if len(r) >= 2:
                    #для відладки
                    print(f"A: {set_a}")
                    print(f"B: {set_b}")
                    print(f"A*B: {cartesian_product}")
                    print(f"S: {s}")
                    print(f"R: {r}")
                    return


#для вікна 4
def get_union():
    return s | r

def get_intersection():
    return s & r
```

```
def get_difference():
    return r - s

def get_complement():
    return cartesian_product - r

def get_inverse():
    return {(b, a) for a, b in s}
```

Вміст файлу LR2.py:

```
import tkinter as tk
import LR2Data as data
from tkinter import messagebox
from tkinter import filedialog

group = 42
group_num = 18

set_a = set()
set_b = set()

class Window1:
    def __init__(self, root):
        self.root = root
        self.root.title("Вікно #1")

        menu = tk.Menu(self.root)
        self.root.config(menu=menu)
        menu.add_command(label="Вікно #2", command=self.open_window2)
        menu.add_command(label="Вікно #3", command=self.open_window3)
        menu.add_command(label="Вікно #4", command=self.open_window4)

        frame_info = tk.Frame(self.root)
        frame_info.pack(pady=10)
        tk.Label(frame_info, text="ПІБ студента: Куліков Максим Миколайович").grid(row=0,
column=0)
        tk.Label(frame_info, text=f"Група: ІО-{group}").grid(row=1, column=0)
        tk.Label(frame_info, text=f"Номер у списку: {group_num}").grid(row=2, column=0)
        tk.Label(frame_info, text=f"Розрахований варіант: {data.calculate_var(group,
group_num)}").grid(row=3, column=0)

    def open_window2(self):
```

```
        new_window = tk.Toplevel(self.root)
        Window2(new_window)

    def open_window3(self):
        new_window = tk.Toplevel(self.root)
        Window3(new_window)

    def open_window4(self):
        new_window = tk.Toplevel(self.root)
        Window4(new_window)

class Window2:
    def __init__(self, root):
        self.root = root
        self.root.title("Вікно #2")
        tk.Label(root, text="Жіночі імена").grid(row=0, column=0)
        self.list_female = tk.Listbox(root, selectmode=tk.MULTIPLE, height=10)
        self.list_female.grid(row=1, column=0, padx=15, pady=10)
        tk.Label(root, text="Чоловічі імена").grid(row=0, column=2)
        self.list_male = tk.Listbox(root, selectmode=tk.MULTIPLE, height=10)
        self.list_male.grid(row=1, column=2, padx=15, pady=10)
        for name in data.female_names:
            self.list_female.insert(tk.END, name)
        for name in data.male_names:
            self.list_male.insert(tk.END, name)

        self.radio_var = tk.StringVar(value="A")
        tk.Radiobutton(root, text="Множина A", variable=self.radio_var, value="A").grid(row=3,
column=0)
        tk.Radiobutton(root, text="Множина B", variable=self.radio_var, value="B").grid(row=3,
column=2)
        btn_add = tk.Button(root, text="Додати", command=self.add_to_set)
        btn_add.grid(row=4, column=1, pady=5)
        btn_save_a = tk.Button(root, text="Зберегти A", command=lambda: self.save_set("A"))
        btn_save_a.grid(row=5, column=0)
        btn_save_b = tk.Button(root, text="Зберегти B", command=lambda: self.save_set("B"))
        btn_save_b.grid(row=6, column=0)
        btn_load_a = tk.Button(root, text="Зчитати A", command=lambda: self.load_set("A"))
        btn_load_a.grid(row=5, column=1)
```

```
        btn_load_b = tk.Button(root, text="Зчитати B", command=lambda: self.load_set("B"))
        btn_load_b.grid(row=6, column=1)
        btn_clear_a = tk.Button(root, text="Очистити A", command=lambda: self.clear_set("A"))
        btn_clear_a.grid(row=5, column=2)
        btn_clear_b = tk.Button(root, text="Очистити B", command=lambda: self.clear_set("B"))
        btn_clear_b.grid(row=6, column=2, pady=10)

        tk.Label(root, text="Множина A").grid(row=7, column=0)
        self.list_a = tk.Listbox(root, height=10)
        self.list_a.grid(row=8, column=0, padx=15, pady=10)
        tk.Label(root, text="Множина B").grid(row=7, column=2)
        self.list_b = tk.Listbox(root, height=10)
        self.list_b.grid(row=8, column=2, padx=15, pady=10)

    def add_to_set(self):
        selected_names = [self.list_female.get(i) for i in self.list_female.curselection()]
        selected_names += [self.list_male.get(i) for i in self.list_male.curselection()]
        target_set = data.set_a if self.radio_var.get() == "A" else data.set_b
        target_list = self.list_a if self.radio_var.get() == "A" else self.list_b

        for name in selected_names:
            if name not in target_set:
                target_set.add(name)
                target_list.insert(tk.END, name)

    def clear_set(self, which):
        if which == "A":
            data.set_a.clear()
            self.list_a.delete(0, tk.END)
            messagebox.showinfo("Успіх!", "Множину A очищено!")
        else:
            data.set_b.clear()
            self.list_b.delete(0, tk.END)
            messagebox.showinfo("Успіх!", "Множину B очищено!")

    def save_set(self, which):
        target_set = data.set_a if which == "A" else data.set_b
        file_path = filedialog.asksaveasfilename(defaultextension=".txt", filetypes=[("Text files",
"*.txt")])
```

```
        if file_path:
            with open(file_path, "w") as file:
                for item in target_set:
                    file.write(f"{item}\n")
            print(f"Множина {which} збережена у файл {file_path}")

    def load_set(self, which):
        file_path = filedialog.askopenfilename(filetypes=[("Text files", "*.txt")])
        if file_path:
            with open(file_path, "r") as file:
                names = file.readlines()
                names = [name.strip() for name in names]
                target_set = data.set_a if which == "A" else data.set_b
                target_list = self.list_a if which == "A" else self.list_b
                target_set.clear()
            for name in names:
                if name not in target_set:
                    target_set.add(name)
                    target_list.insert(tk.END, name)
            messagebox.showinfo("Успіх!", "Множина успішно завантажена!")

class Window3:
    def __init__(self, root):
        self.root = root
        self.root.title("Вікно #3")
        self.root.geometry("1200x470")

        tk.Label(root, text="Множина A:").place(x=20, y=30)
        self.list_a = tk.Listbox(root, selectmode=tk.MULTIPLE, height=10)
        tk.Label(root, text="Множина B:").place(x=20, y=230)
        self.list_b = tk.Listbox(root, selectmode=tk.MULTIPLE, height=10)

        self.list_a.place(x=20, y=50)
        self.list_b.place(x=20, y=250)
        self.button_generate = tk.Button(root, text="Згенерувати S і R", command=lambda:
self.calculate_r_s())
        self.button_generate.place(x=20, y=420)
        for item in sorted(data.set_a):
            self.list_a.insert(tk.END, item)
```

```
        for item in sorted(data.set_b):
            self.list_b.insert(tk.END, item)
        tk.Label(root, text="aSb, якщо a мати b.").place(x=180, y=20)
        tk.Label(root, text="aRb, якщо a онука b.").place(x=680, y=20)
        self.canvas_s = tk.Canvas(root, width=500, height=400, bg="white")
        self.canvas_s.place(x=180, y=50)
        self.canvas_r = tk.Canvas(root, width=500, height=400, bg="white")
        self.canvas_r.place(x=680, y=50)
        self.coord_dict_a = self.create_coordinates(data.set_a, 50, 100)
        self.coord_dict_b = self.create_coordinates(data.set_b, 50, 300)
        self.create_labels_and_ovals(self.canvas_s, data.set_a, self.coord_dict_a)
        self.create_labels_and_ovals(self.canvas_s, data.set_b, self.coord_dict_b)
        self.create_labels_and_ovals(self.canvas_r, data.set_a, self.coord_dict_a)
        self.create_labels_and_ovals(self.canvas_r, data.set_b, self.coord_dict_b)

    def calculate_r_s(self):
        data.create_random_s()
        data.create_random_r()
        self.update_graphs()

    def create_coordinates(self, names_set, x_start, y_start):
        coordinates = {}
        count = len(names_set)
        if count == 0:
            return coordinates
        spacing = 300 // max(1, count - 1)
        for idx, name in enumerate(sorted(names_set)):
            coordinates[name] = (x_start + idx * spacing, y_start)
        return coordinates

    def create_labels_and_ovals(self, canvas, names_set, coord_dict):
        for name, (x, y) in coord_dict.items():
            canvas.create_text(x, y + 20, text=name, font='Arial 10')
            canvas.create_oval(x - 5, y - 5, x + 5, y + 5, fill="black")

    def draw_relations(self, canvas, relations, color):
        for a, b in relations:
            if a in self.coord_dict_a and b in self.coord_dict_b:
                x1, y1 = self.coord_dict_a[a]
```

```
            x2, y2 = self.coord_dict_b[b]
            canvas.create_line(x1, y1, x2, y2, arrow="last", fill=color)



    def update_graphs(self):
        self.canvas_s.delete("all")
        self.canvas_r.delete("all")
        self.coord_dict_a = self.create_coordinates(data.set_a, 50, 100)
        self.coord_dict_b = self.create_coordinates(data.set_b, 50, 300)
        self.create_labels_and_ovals(self.canvas_s, data.set_a, self.coord_dict_a)
        self.create_labels_and_ovals(self.canvas_s, data.set_b, self.coord_dict_b)
        self.create_labels_and_ovals(self.canvas_r, data.set_a, self.coord_dict_a)
        self.create_labels_and_ovals(self.canvas_r, data.set_b, self.coord_dict_b)
        self.draw_relations(self.canvas_s, data.s, "blue")
        self.draw_relations(self.canvas_r, data.r, "red")

class Window4:
    def __init__(self, root):
        self.root = root
        self.root.title("Вікно #4")

        self.canvas = tk.Canvas(root, width=600, height=400, bg="white")
        self.canvas.pack(pady=10)

        tk.Button(root, text="Об'єднання (R∪S)",
                  command=lambda: self.draw_result(set(data.get_union()), "blue")).pack()
        tk.Button(root, text="Перетин (R∩S)",
                  command=lambda: self.draw_result(set(data.get_intersection()), "green")).pack()
        tk.Button(root, text="Різниця (R\\S)",
                  command=lambda: self.draw_result(set(data.get_difference()), "red")).pack()
        tk.Button(root, text="Доповнення (U\\R)",
                  command=lambda: self.draw_result(set(data.get_complement()), "purple")).pack()
        tk.Button(root, text="Обернене (S⁻¹)",
                  command=lambda: self.draw_result(set(data.get_inverse()), "orange")).pack()

    def draw_result(self, relations, color):
        self.canvas.delete("all")

        coord_a = self.create_coordinates(data.set_a, 50, 100)
```

```
        coord_b = self.create_coordinates(data.set_b, 50, 300)

        self.draw_nodes(coord_a)
        self.draw_nodes(coord_b)
        if color == "orange":
            self.draw_relations(relations, coord_b, coord_a, color)
        else:
            self.draw_relations(relations, coord_a, coord_b, color)

    def create_coordinates(self, names_set, x_start, y_start):
        coordinates = {}
        count = len(names_set)
        spacing = 500 // max(1, count - 1) if count > 1 else 250

        for idx, name in enumerate(sorted(names_set)):
            coordinates[name] = (x_start + idx * spacing, y_start)
        return coordinates

    def draw_nodes(self, coord_dict):
        for name, (x, y) in coord_dict.items():
            self.canvas.create_text(x, y + 20, text=name, font='Arial 10')
            self.canvas.create_oval(x - 5, y - 5, x + 5, y + 5, fill="black")

    def draw_relations(self, relations, coord_a, coord_b, color):
        for a, b in relations:
            if a in coord_a and b in coord_b:
                x1, y1 = coord_a[a]
                x2, y2 = coord_b[b]
                self.canvas.create_line(x1, y1, x2, y2, arrow="last", fill=color)

if __name__ == "__main__":
    root = tk.Tk()
    app = Window1(root)
    root.mainloop()
```

Результат виконання програми:

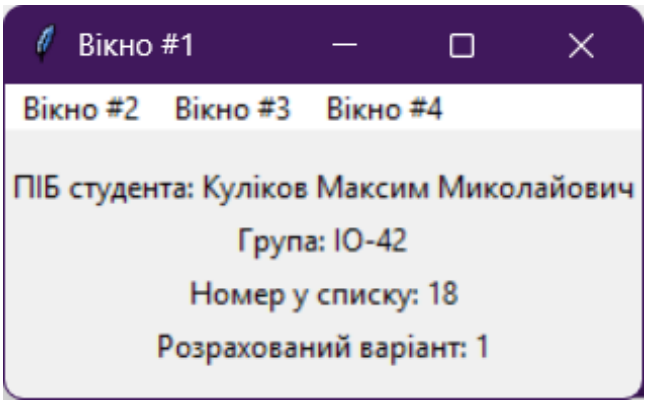

Рисунок 2 – результат виконання програми у вікні 1

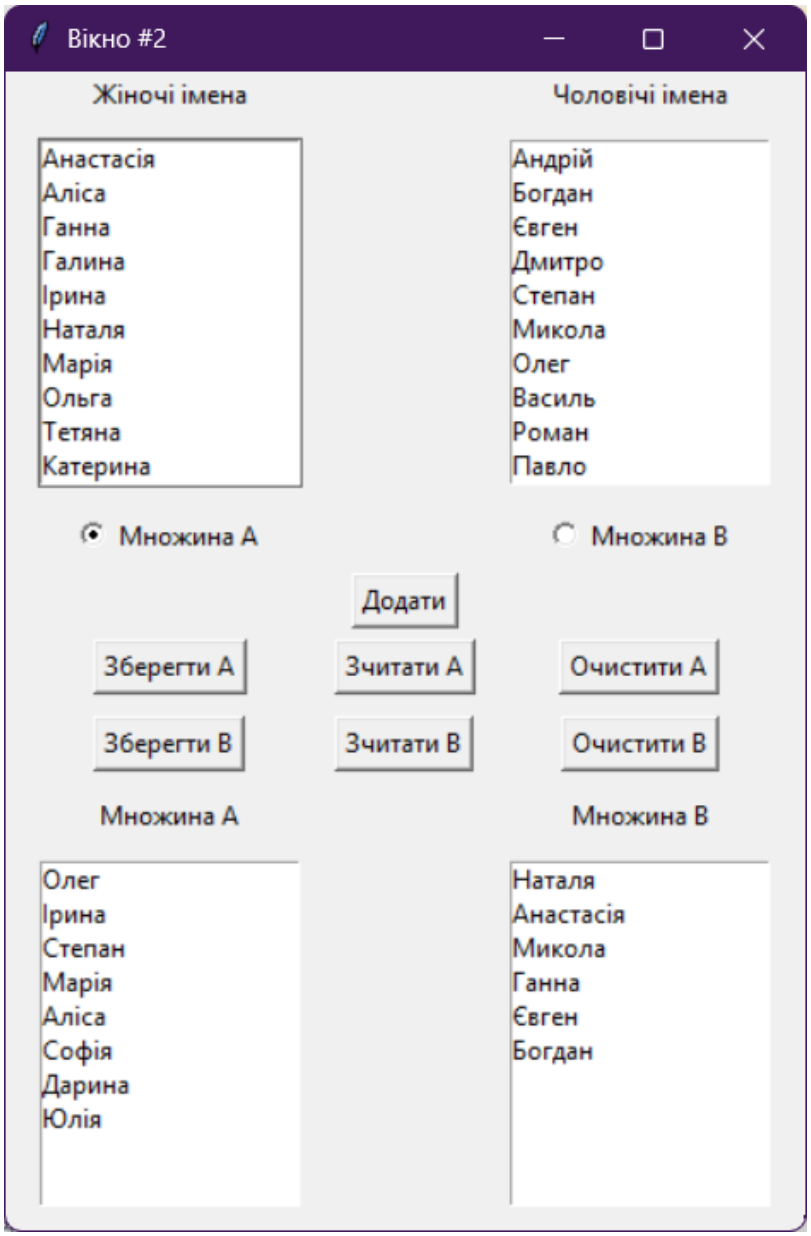

Рисунок 3 – результат виконання програми у вікні 2

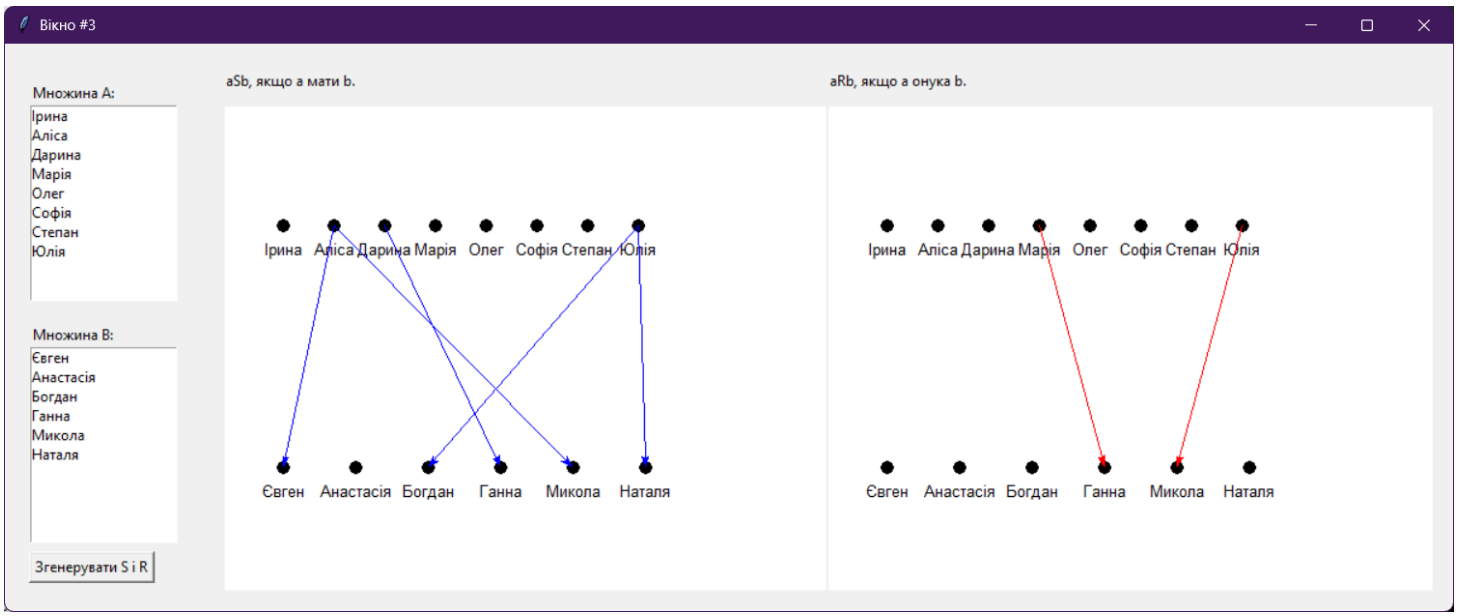

Рисунок 4 – результат виконання програми у вікні 3

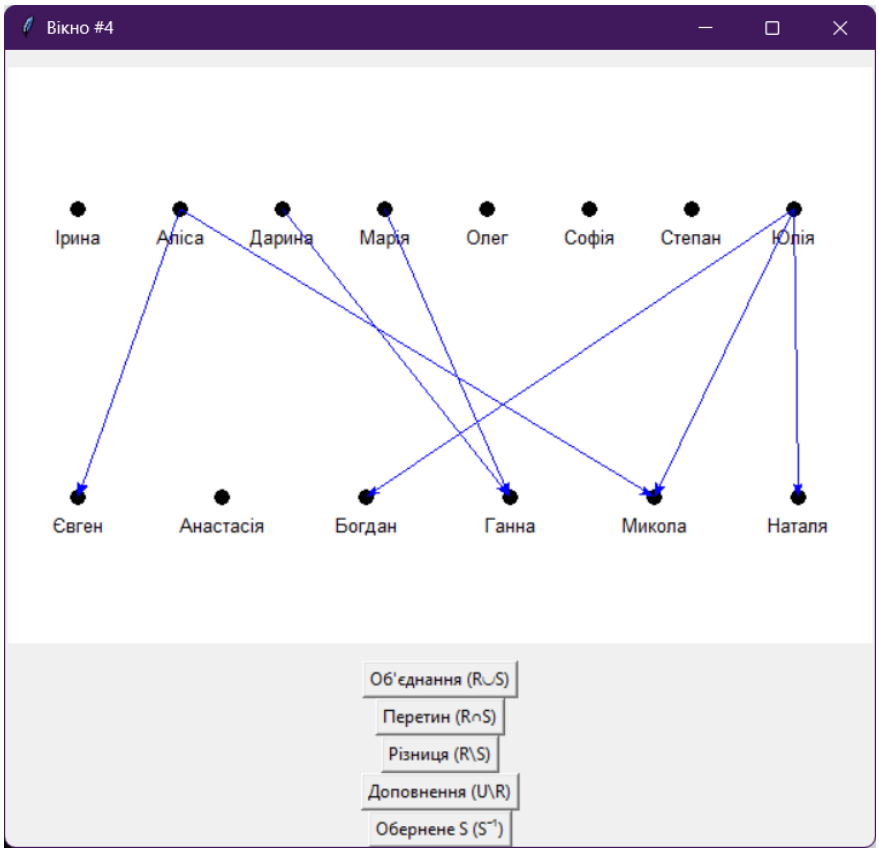

Рисунок 5 – результат виконання програми у вікні 4 (R∪S)

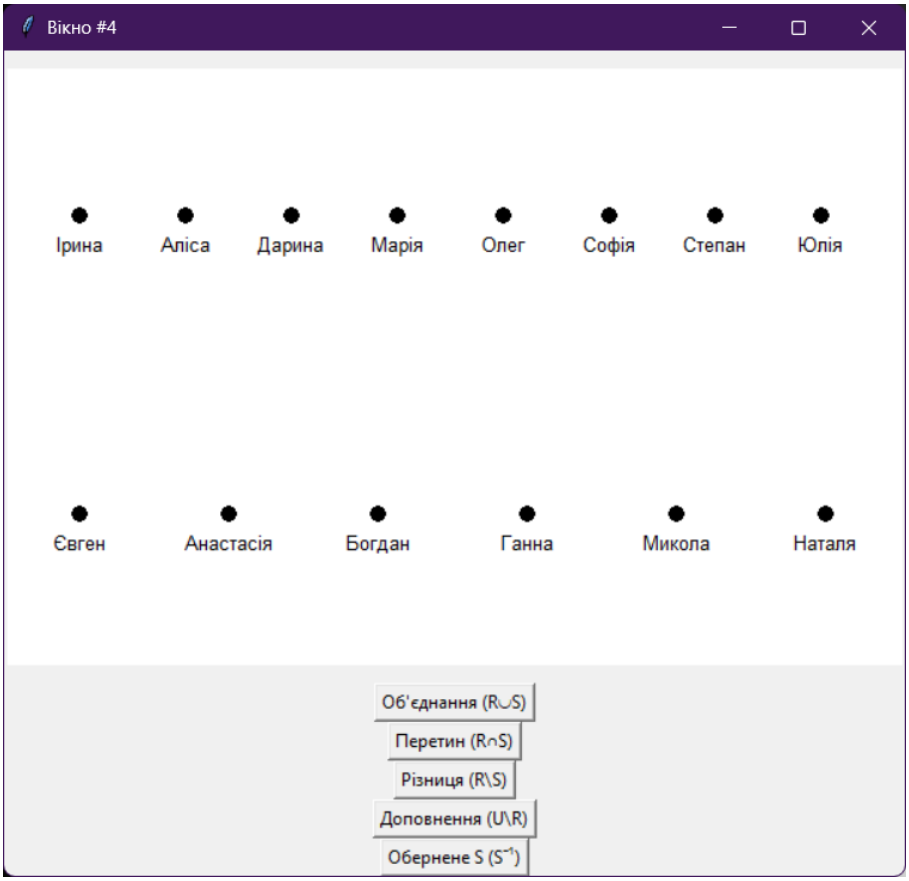

Рисунок 6 – результат виконання програми у вікні 4 (R∩S)

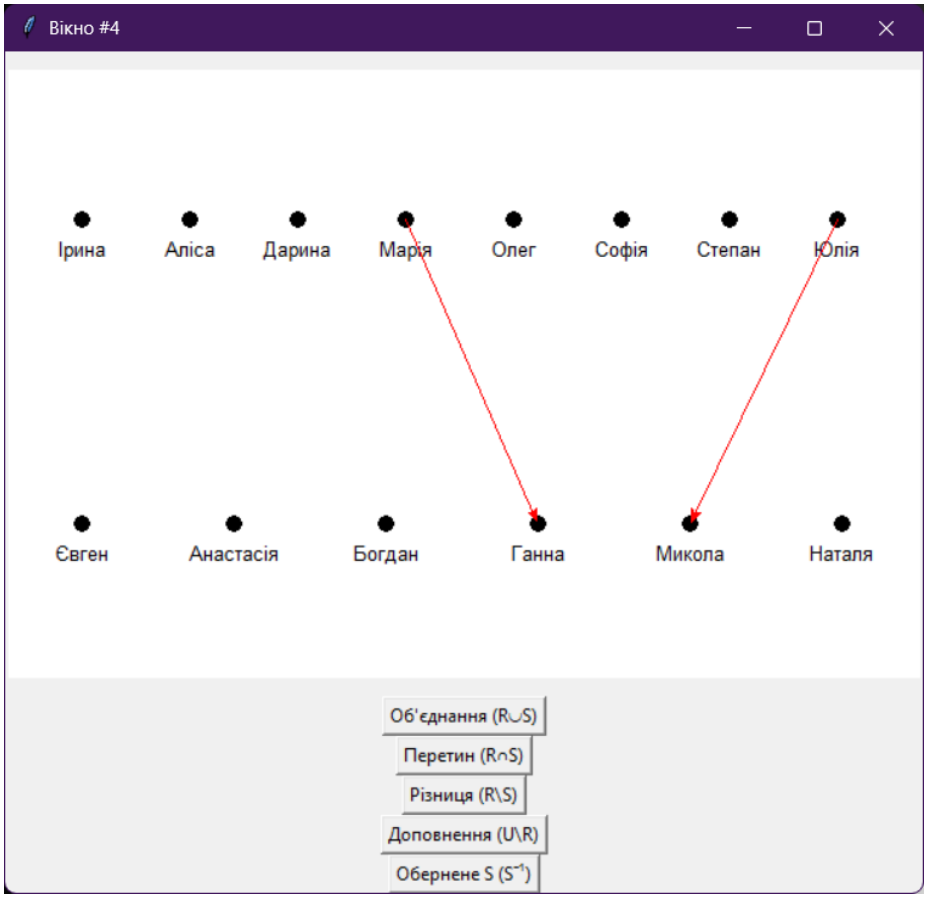

Рисунок 7 – результат виконання програми у вікні 4 (R\S)

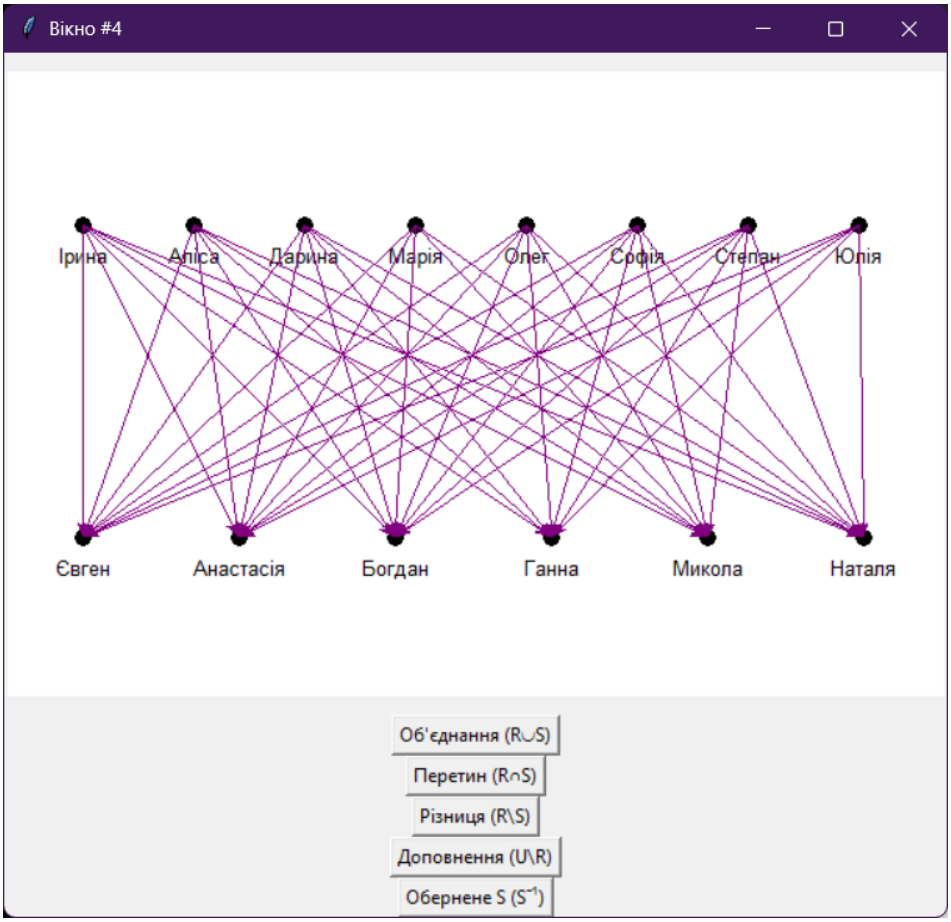

Рисунок 8 – результат виконання програми у вікні 4 (U\R)

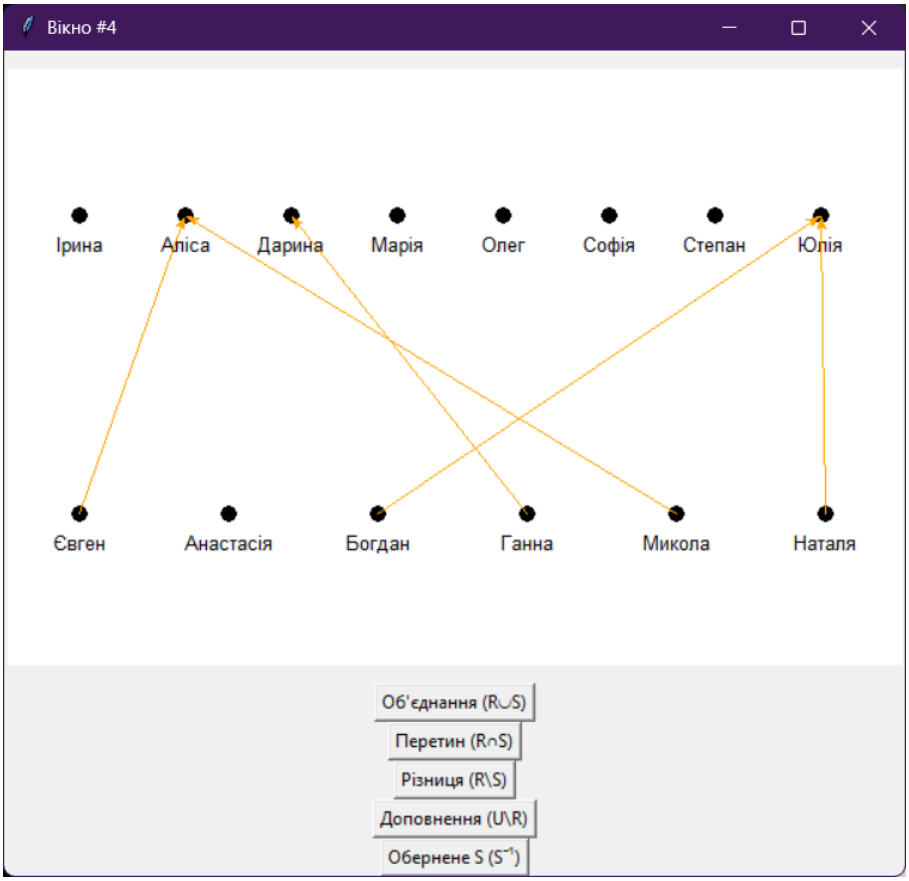

Рисунок 9 – результат виконання програми у вікні 4 (S^{-1})

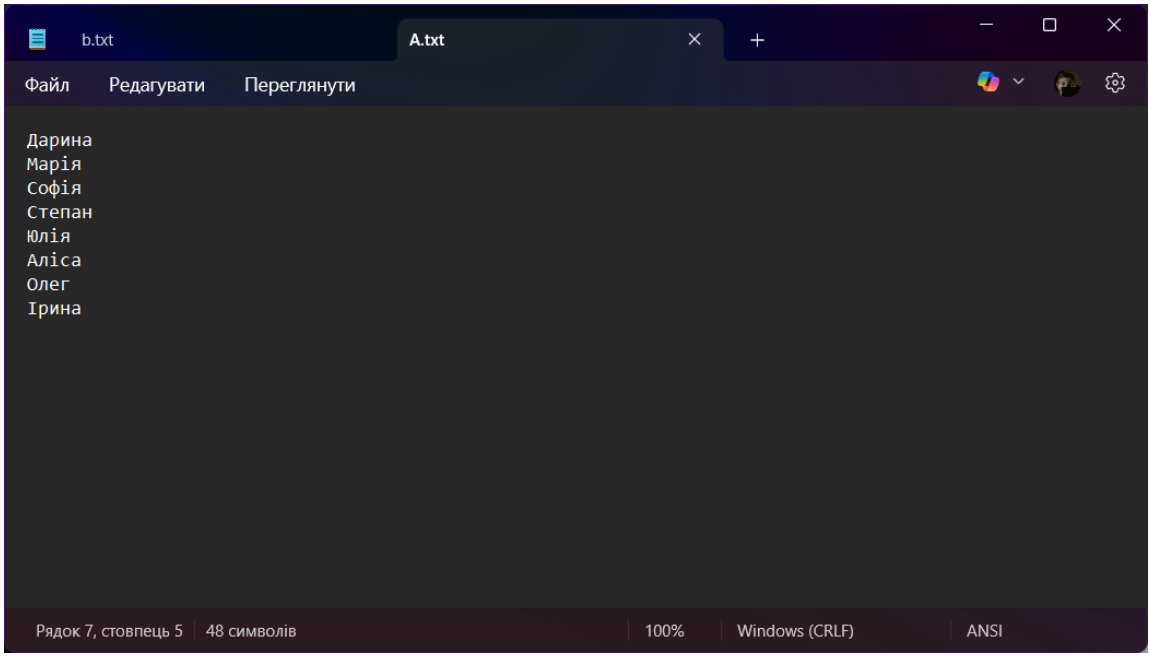

Рисунок 10 – результат збереження множини A у файл

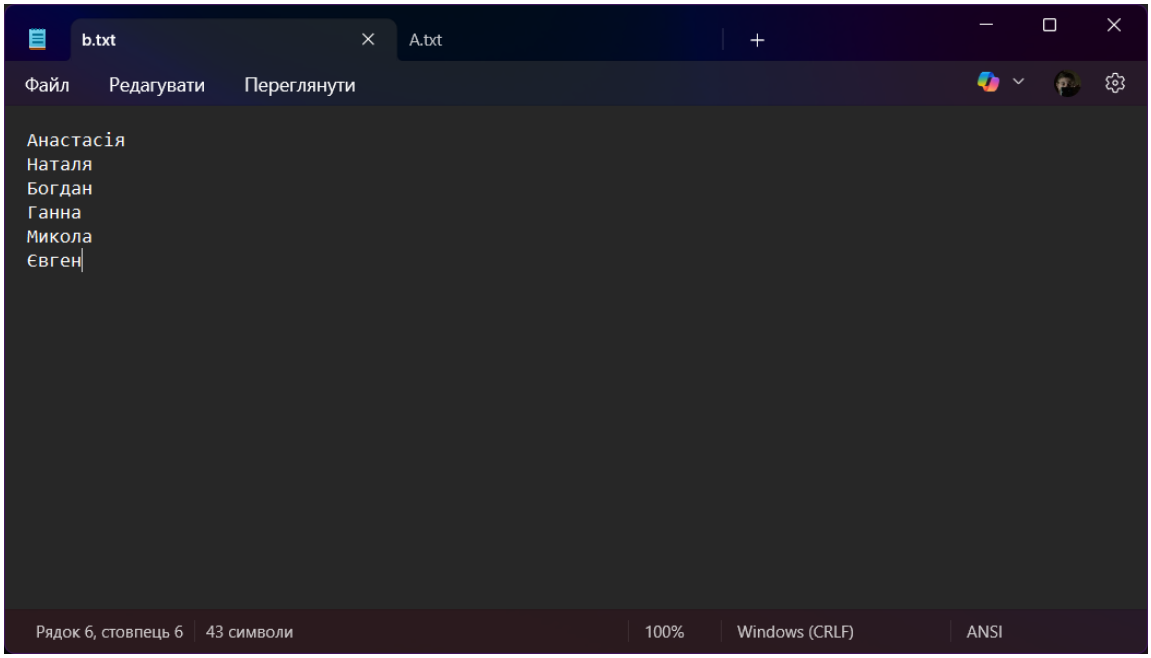

Рисунок 11 – результат збереження множини B у файл

Висновки:

Отже, під час виконання лабораторної роботи я розробив програму, яка генерує відношення відповідно до індивідуального завдання, а також дозволяє візуалізувати їх, зображуючи їх на графах. При встановленні відношення aRb виникли проблеми з нелогічними сімейними зв’язками, зокрема, коли жінка ставала власною онукою. Це виправлено шляхом коригування логіки формування пар, але через це кількість зв’язків у відношенні R стала обмеженою.